

WeberTech Licensing System

Arquitetura de licenciamento offline com chaves assimétricas,
ativação por QR Code e ficheiros de licença .wta

Autor / Arquitetura: Weber Tech

Produtos abrangidos: School Manager · KiVenda Desktop · SmartGest · futuros produtos

Versão do documento: 1.0

Data: Julho de 2026

Índice

1. Contexto e Problema a Resolver
2. Visão Geral da Solução
3. Arquitetura da Solução (.NET)
4. Par de Chaves Assimétricas (RSA)
5. Identificação da Máquina (Machine ID)
6. Fluxo Completo de Ativação Offline (com QR Code)
7. Formato do Pedido de Ativação (QR Code)
8. Formato do Ficheiro de Licença (.wta)
9. ProductType e Perfis de Produto
10. WeberTech.Licensing (Core / SDK) — API Pública
11. WeberTech.LicenseGenerator — Ferramenta do Emissor
12. Integração no Software do Cliente
13. Segurança, Ameaças e Mitigações
14. Distribuição do SDK a Terceiros (Outros Devs)
15. Extensibilidade Futura
16. Roteiro de Implementação Sugerido
17. Conclusão

1. Contexto e Problema a Resolver

A Weber Tech desenvolve vários softwares desktop que correm **offline**, nos computadores dos clientes, sem ligação permanente à internet: **School Manager**, **KiVenda Desktop**, **SmartGest**, e outros que vão surgir. O modelo de negócio exige que cada instalação seja licenciada por cliente, mas o método tradicional de ativação (o técnico deslocar-se fisicamente até ao cliente para libertar o software) tornou-se o maior gargalo operacional à medida que o número de clientes cresce.

O objetivo deste projeto é criar um **sistema de licenciamento próprio, reutilizável entre produtos**, que permita ativar qualquer software offline **sem exigir internet no computador do cliente** e sem deslocação do técnico, mantendo um nível de segurança criptográfica adequado (chaves assimétricas), e que possa, no futuro, ser distribuído como SDK para outros programadores que queiram licenciar os seus próprios produtos.

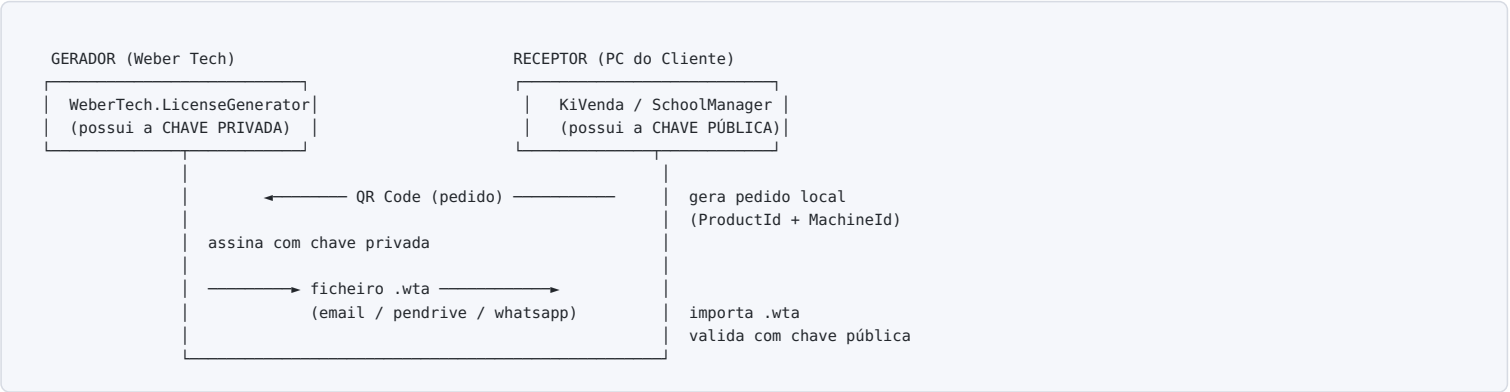
Objetivo em uma frase

Um sistema de duas partes — um *gerador de licenças* (fica na Weber Tech) e um *validador embutido* (vai dentro de cada produto) — que comunicam através de um ficheiro de licença assinado digitalmente (**.wta**), cujo pedido inicial é transmitido através de um QR Code, sem depender de rede.

2. Visão Geral da Solução

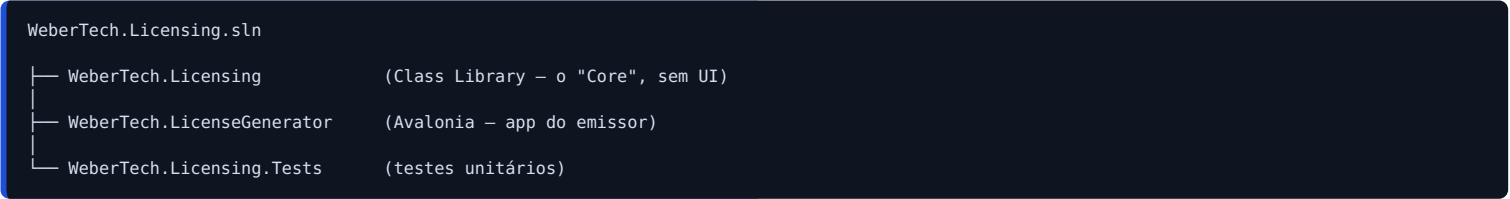
A solução assenta em três pilares:

- Criptografia assimétrica (RSA):** a Weber Tech guarda a *chave privada* — nunca sai do computador do gerador de licenças. Cada software distribuído leva apenas a *chave pública* correspondente, embutida no binário. A chave privada assina; a chave pública só consegue verificar, nunca assinar. Isto significa que mesmo que alguém descompile o software do cliente e extraia a chave pública, não consegue forjar uma licença.
- Ativação sem internet via QR Code:** o software gera, localmente, um "pedido de ativação" que identifica o produto e a máquina. Esse pedido é mostrado como QR Code no ecrã. Não é preciso o computador do cliente ter internet — o QR Code pode ser fotografado com um telemóvel e enviado por WhatsApp, ou o texto pode ser copiado para uma pen. O emissor da licença (Weber Tech) processa esse pedido no **LicenseGenerator** e devolve um ficheiro de licença.
- Ficheiro de licença portátil (.wta):** um ficheiro assinado digitalmente, específico para aquele produto + máquina + plano, que o cliente simplesmente coloca/importa no software. A partir daí, o software valida localmente a assinatura com a chave pública embutida — sem precisar de rede nunca mais.



3. Arquitetura da Solução (.NET)

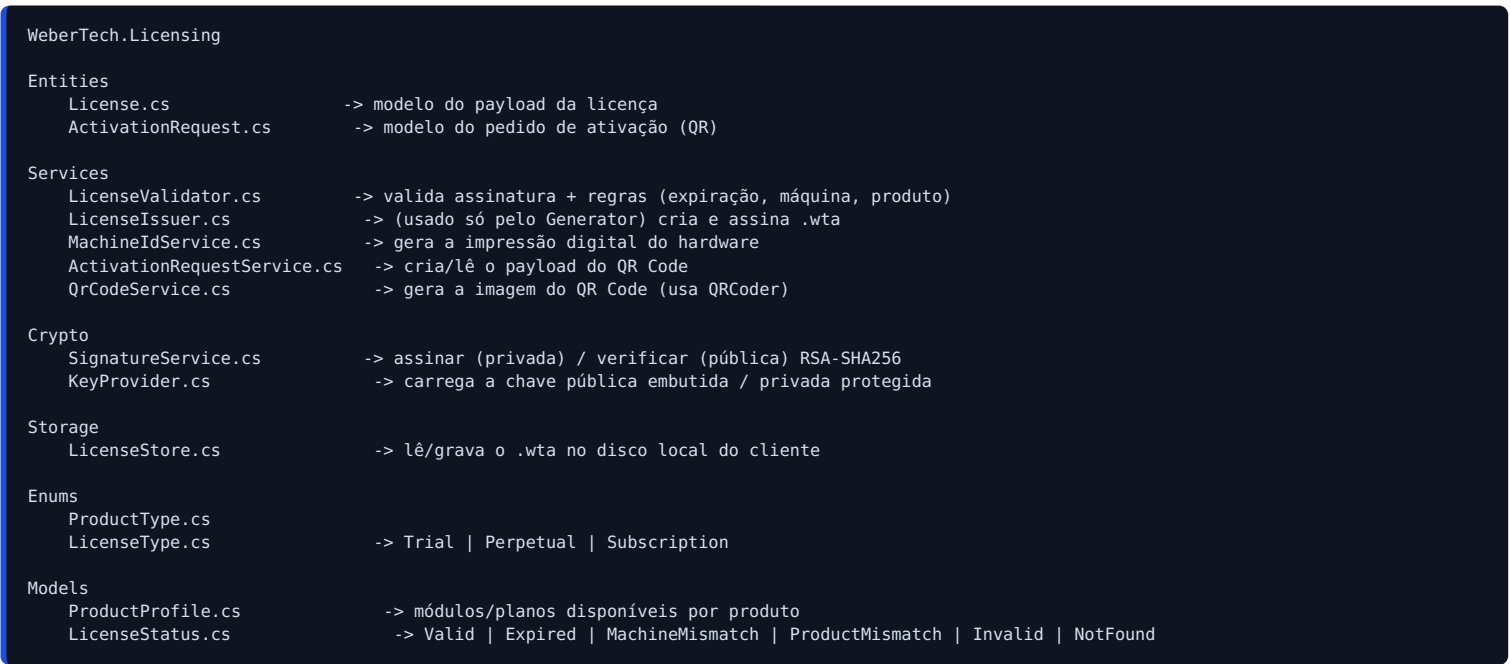
Seguindo a linha do que já discutimos, o licenciamento não deve ser um módulo de um produto — deve ser um **produto interno da Weber Tech**, compartilhado entre todos os softwares:



Cada produto apenas referencia a DLL `WeberTech.Licensing` :



3.1 Estrutura interna do Core



Porquê "WeberTech.Licensing" e não "...Licensing.Core"?

Porque este projeto *já* é o núcleo. Mantemos o nome limpo para crescer naturalmente no futuro: `WeberTech.Licensing.Api` , `WeberTech.Licensing.CLI` , `WeberTech.Licensing.SDK` , sem redundância no nome.

4. Par de Chaves Assimétricas (RSA)

Usa-se RSA de, no mínimo, **2048 bits** (recomendado 3072 bits para maior margem de segurança a longo prazo), com assinatura **RSA-SHA256 (PKCS#1 v1.5 ou PSS)**. O par de chaves é gerado *uma única* vez pela Weber Tech e gerido manualmente — nunca gerado automaticamente pela aplicação do cliente.

4.1 Geração do par de chaves (feita uma vez, offline, pela Weber Tech)

```
using System.Security.Cryptography;

using var rsa = RSA.Create(3072);

// Guardar em local seguro, NUNCA distribuído com o software
File.WriteAllText("wt_private.pem", rsa.ExportPkcs8PrivateKeyPem());

// Esta sim vai embutida em cada produto
File.WriteAllText("wt_public.pem", rsa.ExportSubjectPublicKeyInfoPem());
```

4.2 Onde fica cada chave

Chave	Onde vive	Função
Privada	Só no computador/cofre da Weber Tech, usada pelo <code>LicenseGenerator</code>	Assinar licenças (.wta)
Pública	Embutida como recurso (<code>Resource</code>) dentro de <code>WeberTech.Licensing</code> , compilada em cada produto	Verificar assinaturas

Nota de segurança

A chave privada nunca deve ser gravada em texto simples num repositório Git, nem sequer no projeto do `LicenseGenerator` . Recomenda-se guardá-la protegida por password (PEM cifrado) ou num cofre local (ex.: Windows DPAPI), carregada apenas em tempo de execução do gerador.

4.3 Assinar (no Generator) e Validar (no produto do cliente)

```
// Assinar (Generator, tem a privada)
byte[] payloadBytes = Encoding.UTF8.GetBytes(jsonPayload);
byte[] signature = rsaPrivate.SignData(payloadBytes, HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);

// Validar (produto do cliente, só tem a pública)
bool valido = rsaPublic.VerifyData(payloadBytes, signature, HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);
```

5. Identificação da Máquina (Machine ID)

Cada licença é presa a uma máquina específica, para impedir que um ficheiro `.wta` seja copiado para outro computador. O `MachineIdService` gera um identificador estável, calculado a partir de características de hardware que raramente mudam:

- Número de série da placa-mãe (`Win32_BaseBoard.SerialNumber`)
- Identificador único do processador (`Win32_Processor.ProcessorId`)
- Número de série do disco do sistema (`Win32_DiskDrive.SerialNumber`)

Estes três valores são concatenados e passados por `SHA-256` , produzindo um Machine ID curto e determinístico. Este processo corre inteiramente no computador do cliente, sem qualquer chamada externa.

```
public string GetMachineId()
{
    string cpuId  = QueryWmi("Win32_Processor", "ProcessorId");
    string boardId = QueryWmi("Win32_BaseBoard", "SerialNumber");
    string diskId  = QueryWmi("Win32_DiskDrive", "SerialNumber");

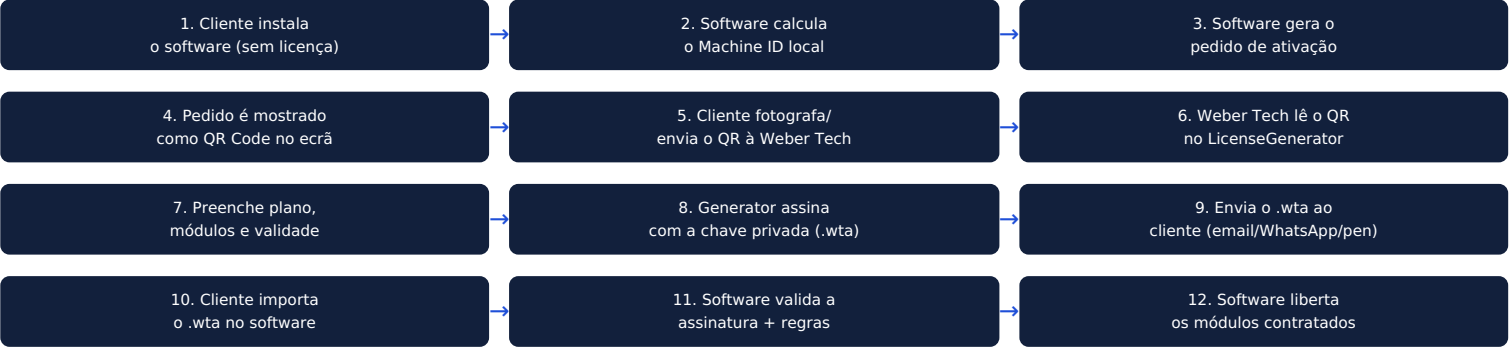
    string raw = $"{cpuId}|{boardId}|{diskId}";
    byte[] hash = SHA256.HashData(Encoding.UTF8.GetBytes(raw));
    return Convert.ToString(hash, 2); // 32 caracteres, suficiente
}
```

Tolerância a pequenas mudanças

Trocar um disco externo ou uma impressora não afeta o Machine ID, porque só usamos o disco de sistema, a CPU e a placa-mãe — as três peças que definem "esta é fisicamente a mesma máquina". Trocar a motherboard, sim, vai gerar um novo Machine ID; nesse caso, o cliente simplesmente pede uma nova ativação (novo QR Code → novo `.wta`).

6. Fluxo Completo de Ativação Offline (com QR Code)

Este é o percurso ponta-a-ponta, do momento em que o software é instalado até estar licenciado:



Repara que em nenhum momento o computador do cliente precisa de internet — só precisa de uma forma qualquer (fora do software) de *enviar* o QR Code e de *receber* o ficheiro `.wta` de volta. O telemóvel do próprio cliente resolve os dois lados.

7. Formato do Pedido de Ativação (QR Code)

O QR Code não contém a licença — contém apenas um **pedido**, com informação suficiente para o gerador saber para que produto, versão e máquina deve emitir a licença. É um JSON compacto, comprimido com Base64, prefixado por uma marca de formato para facilitar a leitura futura por outras ferramentas:

```
{
  "fmt": "WTAREQ1",
  "productId": "kivenda.desktop_v03",
  "machineId": "9F2C7A1E4B6D0083",
  "requestId": "b13e1a2a-8e3d-4a91-9f2b-6b6a9e6a2e10",
  "requestedAt": "2026-07-30T09:15:00Z"
}
```

Este JSON é convertido para Base64 e colocado dentro do QR Code. O campo `productId` segue o padrão `<produto>_<versão>` pedido — por exemplo `kivenda.desktop_v03`, `schoolmanager.desktop_v01`. É este mesmo texto que identifica, no `LicenseGenerator`, para que `ProductProfile` a licença deve ser gerada.

```
// Lado do produto (cliente): gerar o QR
var request = new ActivationRequest
{
    ProductId    = "kivenda.desktop_v03",
    MachineId    = machineIdService.GetMachineId(),
    RequestId    = Guid.NewGuid(),
    RequestedAt  = DateTime.UtcNow
};

string json    = JsonSerializer.Serialize(request);
string base64  = Convert.ToBase64String(Encoding.UTF8.GetBytes(json));
string qrText  = $"WTAREQ1:{base64}";

qrCodeService.RenderToScreen(qrText); // usa QRCode
```

Como o Generator "lê" o QR

O `LicenseGenerator` pode ler o QR de duas formas: (a) apontando a webcam do PC da Weber Tech ao ecrã do cliente ou a uma foto recebida (usando `ZXing.Net`), ou (b) simplesmente colando o texto Base64 recebido por WhatsApp/email numa caixa de texto. A segunda opção é a mais prática no dia-a-dia e não exige câmara nenhuma.

8. Formato do Ficheiro de Licença (.wta)

O ficheiro `.wta` ("WeberTech Activation") é um envelope JSON assinado digitalmente. Contém o payload da licença, a assinatura RSA desse payload, e metadados sobre a própria chave/algoritmo usados — para permitir rotação de chaves no futuro.

8.1 Payload da licença

```
{
  "licenseId": "6a4f9e0a-1d2b-4a3f-9c7e-0a1b2c3d4e5f",
  "productId": "kivenda.desktop_v03",
  "customerId": "00045",
  "customerName": "Loja Central Lda",
  "machineId": "9F2C7A1E4B6D0083",
  "plan": "professional",
  "type": "Subscription",
  "features": ["Caixa", "Estoque", "Compras", "Vendas", "Relatorios"],
  "issuedAt": "2026-07-30T10:00:00Z",
  "expiresAt": "2027-07-30T10:00:00Z"
}
```

8.2 Envelope final gravado como .wta

```
{
  "payload": "eyJsaWNlbnNlSWQiOiI2YT1r0Xg3ZP0j5f3aQe4v...",
  "signature": "MC0CF0Cd2G+7hZ1r0Xg3ZP0j5f3aQe4v...",
  "algorithm": "RSA-SHA256",
  "keyVersion": 1
}
```

Note que o `payload` vai em Base64 dentro do envelope — não em texto simples — apenas para evitar que um utilizador edite manualmente o JSON num editor de texto comum sem perceber que está a invalidar a assinatura. Isto não é segurança criptográfica (a assinatura é que garante isso), é apenas fricção adicional.

8.3 Emitir e gravar o .wta (no Generator)

```
string payloadJson = JsonSerializer.Serialize(license);
string payloadB64 = Convert.ToBase64String(Encoding.UTF8.GetBytes(payloadJson));

byte[] signature = rsaPrivate.SignData(
    Encoding.UTF8.GetBytes(payloadJson),
    HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);

var envelope = new LicenseFile
{
    Payload = payloadB64,
    Signature = Convert.ToBase64String(signature),
    Algorithm = "RSA-SHA256",
    KeyVersion = 1
};

File.WriteAllText("Loja_Central_KiVenda.wta", JsonSerializer.Serialize(envelope));
```

8.4 Validar o .wta (no produto do cliente)

```
var envelope = JsonSerializer.Deserialize<LicenseFile>(File.ReadAllText(path));
byte[] payloadBytes = Convert.FromBase64String(envelope.Payload);
byte[] signature = Convert.FromBase64String(envelope.Signature);

bool assinaturaValida = rsaPublic.VerifyData(
    payloadBytes, signature, HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);

if (!assinaturaValida)
    return LicenseStatus.Invalid;

var license = JsonSerializer.Deserialize<License>(Encoding.UTF8.GetString(payloadBytes));

if (license.ProductId != currentProduct.Id) return LicenseStatus.ProductMismatch;
if (license.MachineId != currentMachineId) return LicenseStatus.MachineMismatch;
if (license.ExpiresAt < DateTime.UtcNow) return LicenseStatus.Expired;

return LicenseStatus.Valid;
```

9. ProductType e Perfis de Produto

Para que o mesmo `LicenseGenerator` sirva todos os produtos sem confusões (por exemplo, gerar uma licença do KiVenda com o módulo "Alunos", que só existe no School Manager), cada produto tem um **perfil** que descreve os seus módulos e planos disponíveis:

```
public enum ProductType
{
    SchoolManager,
    SmartGest,
    KiVenda
}

public class ProductProfile
{
    public string ProductId { get; set; }           // ex.: "kivenda.desktop"
    public string Name { get; set; }
    public string[] AvailableFeatures { get; set; }
    public string[] AvailablePlans { get; set; }
}
```

Produto	Módulos disponíveis
School Manager	Alunos, Propinas, Financeiro, Relatórios
SmartGest	Contabilidade, IVA/Impostos, Inventário, Relatórios
KiVenda	Caixa, Estoque, Compras, Vendas

Quando o operador seleciona o produto na interface do `LicenseGenerator`, a lista de módulos apresentada muda automaticamente conforme o perfil — tornando impossível emitir uma combinação inválida.

10. WeberTech.Licensing (Core / SDK) — API Pública

Esta é a superfície que qualquer produto (interno ou de terceiros) usa para se integrar. A ideia é que a integração seja de poucas linhas:

```
// No arranque da aplicação
Licensing.Initialize(ProductType.KiVenda, "kivenda.desktop_v03");

var status = Licensing.CurrentStatus; // LicenseStatus

if (status != LicenseStatus.Valid)
{
    // mostra o ecrã de ativação com o QR Code
    var qr = Licensing.GenerateActivationQrCode();
    ShowActivationScreen(qr);
    return;
}

// Verificar módulos contratados
if (Licensing.HasFeature("Relatorios"))
{
    HabilitarMenuRelatorios();
}

int diasRestantes = Licensing.DaysUntilExpiration();
```

Membro	Descrição
Initialize(ProductType, productId)	Carrega a chave pública embutida e tenta ler o .wta local
CurrentStatus	Devolve o LicenseStatus atual
GenerateActivationQrCode()	Gera a imagem do QR Code de pedido de ativação
ImportLicenseFile(path)	Importa e valida um novo .wta escolhido pelo utilizador
HasFeature(string)	Verifica se o módulo consta da licença ativa
DaysUntilExpiration()	Dias restantes até expirar (ou -1 se perpétua)
GetLicenseInfo()	Devolve os dados da licença para mostrar ao utilizador (cliente, plano, validade)

11. WeberTech.LicenseGenerator — Ferramenta do Emissor

Aplicação Avalonia de uso interno da Weber Tech, com a chave privada. Fluxo de utilização:

Passo 1 – Ler o pedido

[Colar código Base64 do QR] [Ler webcam] |

↓ (decodifica productId + machineId automaticamente)

Passo 2 – Confirmar dados do cliente

Produto: ● KiVenda ○ SmartGest ○ School Manager |
Cliente: [Loja Central Lda] |
MachineId: 9F2C7A1E4B6D0083 (auto) |
Plano: [Professional ▼] |
Tipo: ● Assinatura ○ Perpétua ○ Trial |
Início: [30/07/2026] Fim: [30/07/2027] |
Módulos: ☑ Caixa ☑ Estoque ☑ Compras ☑ Vendas |

Passo 3 – Emitir

[Gerar Licença .wta] |

↓

Loja_Central_KiVenda.wta (pronto a enviar ao cliente)

Estrutura de pastas sugerida para o projeto Avalonia:

```
WeberTech.LicenseGenerator
Views/                -> ActivationView, IssueLicenseView, HistoryView
ViewModels/
Services/             -> QrReaderService (ZXing), FileDialogService
Assets/
```

Um detalhe útil: o `LicenseGenerator` deve manter um **histórico local** (uma pequena base SQLite, por exemplo) de todas as licenças já emitidas — por cliente, produto e data — para permitir reemitir facilmente uma licença perdida ou renovar uma assinatura sem repetir todo o processo do QR Code.

12. Integração no Software do Cliente

Para o programador que está a integrar a biblioteca num produto existente, o processo resume-se a três pontos de contacto:

1. **Arranque:** chamar `Licensing.Initialize(...)` antes de mostrar a janela principal.
2. **Ecrã de bloqueio:** se `CurrentStatus != Valid` , mostrar uma janela simples com o QR Code, um botão "Já tenho um ficheiro de licença" (que chama `ImportLicenseFile`), e uma explicação curta de como proceder.
3. **Controlo de funcionalidades:** nos pontos onde um módulo pago é usado, verificar `Licensing.HasFeature("X")` antes de habilitar o menu/ecrã correspondente.

Onde fica o .wta no disco do cliente?

Sugestão: `%ProgramData%\WeberTech\{ProductId}\license.wta` — uma pasta comum a todos os utilizadores da máquina, fora da pasta de instalação (para sobreviver a reinstalações/atualizações do software).

13. Segurança, Ameaças e Mitigações

Ameaça	Mitigação
Copiar o .wta para outro PC	Assinatura cobre o <code>machineId</code> ; noutra máquina o MachineId calculado não bate, licença rejeitada
Editar o payload à mão (mudar data de expiração)	Qualquer alteração ao payload invalida a assinatura RSA — impossível sem a chave privada
Recuar o relógio do sistema para "renovar" um trial	Guardar, junto ao .wta, um carimbo da última verificação bem-sucedida; se a hora atual for anterior a esse carimbo, assumir adulteração
Extrair a chave pública e gerar licenças falsas	Impossível — a chave pública só verifica, não assina; sem a privada não se cria uma assinatura válida
Decompilar o software e "saltar" a verificação no código	Ofuscação do binário (ex.: ConfuserEx, .NET Reactor) + verificação em múltiplos pontos do código, não só no arranque
Reutilizar o mesmo pedido de ativação (QR) várias vezes	Guardar histórico de <code>requestId</code> já emitidos no Generator; opcional para v1, mais relevante se/quando existir revogação online

Sobre revogação

Como o sistema é 100% offline, não existe forma de "desligar" remotamente uma licença já entregue (não há telefonar para casa). Isto é aceitável para a maioria dos casos de uso deste tipo de produto, mas deve ser comunicado como limitação consciente do modelo — e resolvido, se necessário, através do mecanismo online descrito na secção 15.

14. Distribuição do SDK a Terceiros (Outros Devs)

Para permitir que outros programadores (fora da Weber Tech) usem o mesmo sistema nos seus próprios produtos, o `WeberTech.Licensing` deve ser embalado como um **pacote NuGet privado**:

- Publicar num feed NuGet privado (Azure Artifacts, GitHub Packages, ou um feed próprio) — nunca no NuGet.org público, já que a chave pública embutida e a lógica de validação são específicas da Weber Tech.
- Cada novo produto/cliente-parceiro recebe o seu próprio `ProductId` e é registado como um novo `ProductProfile` no `LicenseGenerator` — sem precisar de recompilar o Core.
- Documentação de integração reduzida a: instalar o pacote, chamar `Initialize`, tratar o ecrã de ativação. Tudo o resto (criptografia, QR, machine id) fica escondido dentro do SDK.

15. Extensibilidade Futura

A arquitetura foi pensada para crescer sem quebrar o que já existe:

- `WeberTech.Licensing.Api` — quando fizer sentido, uma API online opcional para emissão remota de licenças (útil para clientes que já têm internet estável) e para uma futura lista de revogação.
- `WeberTech.Licensing.CLI` — ferramenta de linha de comandos para emitir licenças em lote (ex.: renovações anuais de dezenas de clientes de uma vez).
- `WeberTech.Licensing.SDK` — pacote NuGet público/semi-público, com documentação para parceiros externos.
- Suporte a **modo híbrido**: produtos que tenham internet ocasional podem, no arranque, tentar validar online (lista de revogação) e cair de volta para a validação offline por assinatura caso não haja rede — sem alterar o formato do .wta.

16. Roteiro de Implementação Sugerido

Fase	Entregável
1	Gerar o par de chaves RSA definitivo; criar o projeto <code>WeberTech.Licensing</code> com as entidades e o <code>SignatureService</code>
2	Implementar <code>MachineIdService</code> e <code>ActivationRequestService</code> + geração de QR (QRCoder)
3	Implementar <code>LicenseIssuer</code> , <code>LicenseValidator</code> e o formato .wta completo
4	Construir o <code>WeberTech.LicenseGenerator</code> em Avalonia (leitura de QR, formulário, histórico local)
5	Integrar no KiVenda Desktop como piloto; validar o fluxo ponta-a-ponta com um cliente real
6	Replicar a integração em School Manager e SmartGest
7	Empacotar como NuGet privado para abrir a terceiros

17. Conclusão

Este desenho resolve diretamente o gargalo original — a necessidade de deslocação física para ativar software — trocando essa deslocação por uma simples troca de QR Code e ficheiro `.wta`, que pode acontecer inteiramente por WhatsApp ou email, mesmo com o computador do cliente sem internet. Ao mesmo tempo, mantém a arquitetura como um produto interno da Weber Tech, partilhado entre todos os softwares atuais e futuros, e preparado para, mais tarde, ser oferecido como SDK a outros programadores.